Smart Warehouse Inventory using Drones

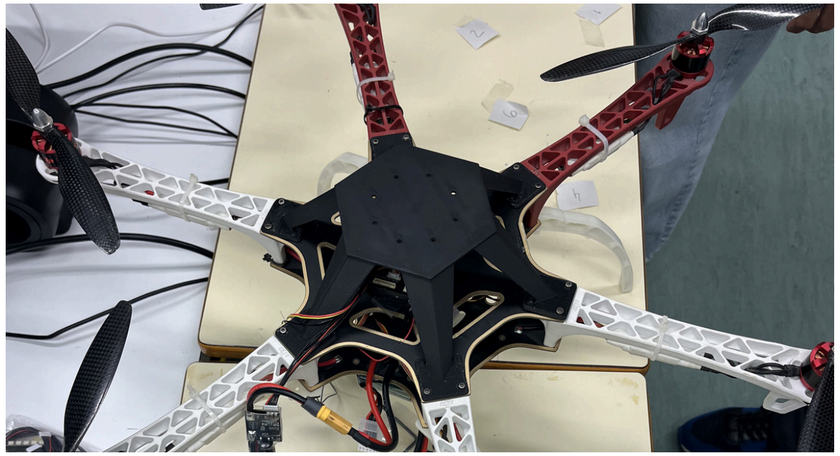
PAE - Advanced Engineering Project

Final Design Review
08 - June - 2026

Table of Contents

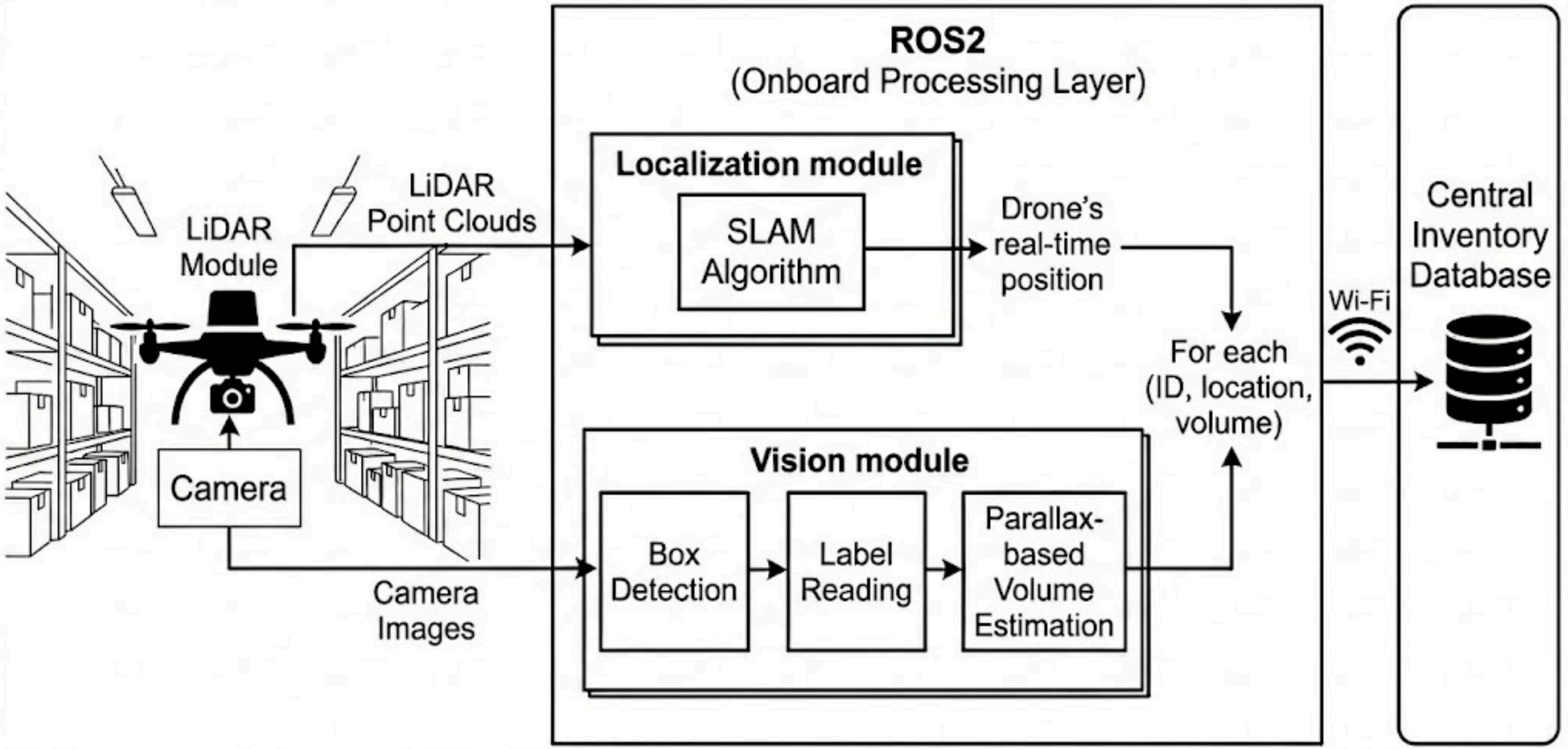
1. Introduction
2. Project scope and objectives
3. WP0: System Integration
4. WP1: Drone control
5. WP1: Lidar
6. WP2: Communication
7. WP3: Visual System
8. Conclusions
9. Demonstration
10. Questions





1. Introduction

System overview: Drone-based warehouse inventory



1. Introduction

Work packages

WP0 **System Integration**

Integration of hardware and software components into a single functional platform.

WP1 **Drone Control and Lidar**

Physical assembly, flight controller tuning, and implementing LiDAR-based SLAM algorithms for autonomous indoor navigation.

WP2 **Communication**

It manages the real-time data flow and Wi-Fi infrastructure between the drone and the central inventory database.

WP3 **Visual System**

Image processing for box detection, label reading, and volume estimation.

2.Project scope and objectives

Project objective:

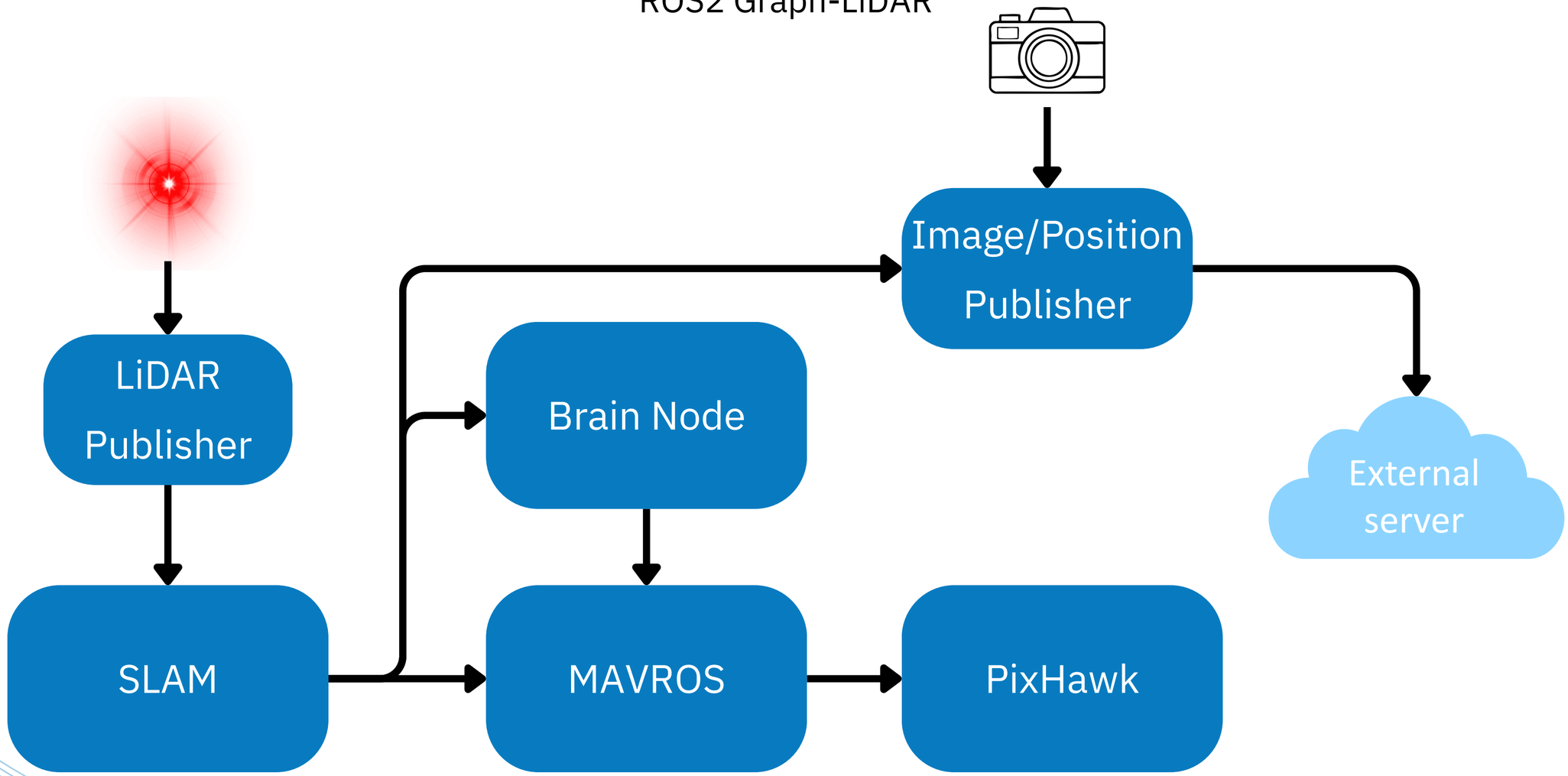
To perform warehouse inventory using drones in order to obtain reliable information in a more efficient and fully automated way.

Project requirements:

- Autonomous flight
- Smart scanning and identification of labels in barcode or QR code format.
- Volumetric analysis to identify materials and estimate stock quantities.
- Inventory results sent to a database for subsequent analysis.
- Warehouse plan available showing the layout of the aisles and locations.
- Wi-Fi network available in the warehouse for data transmission.

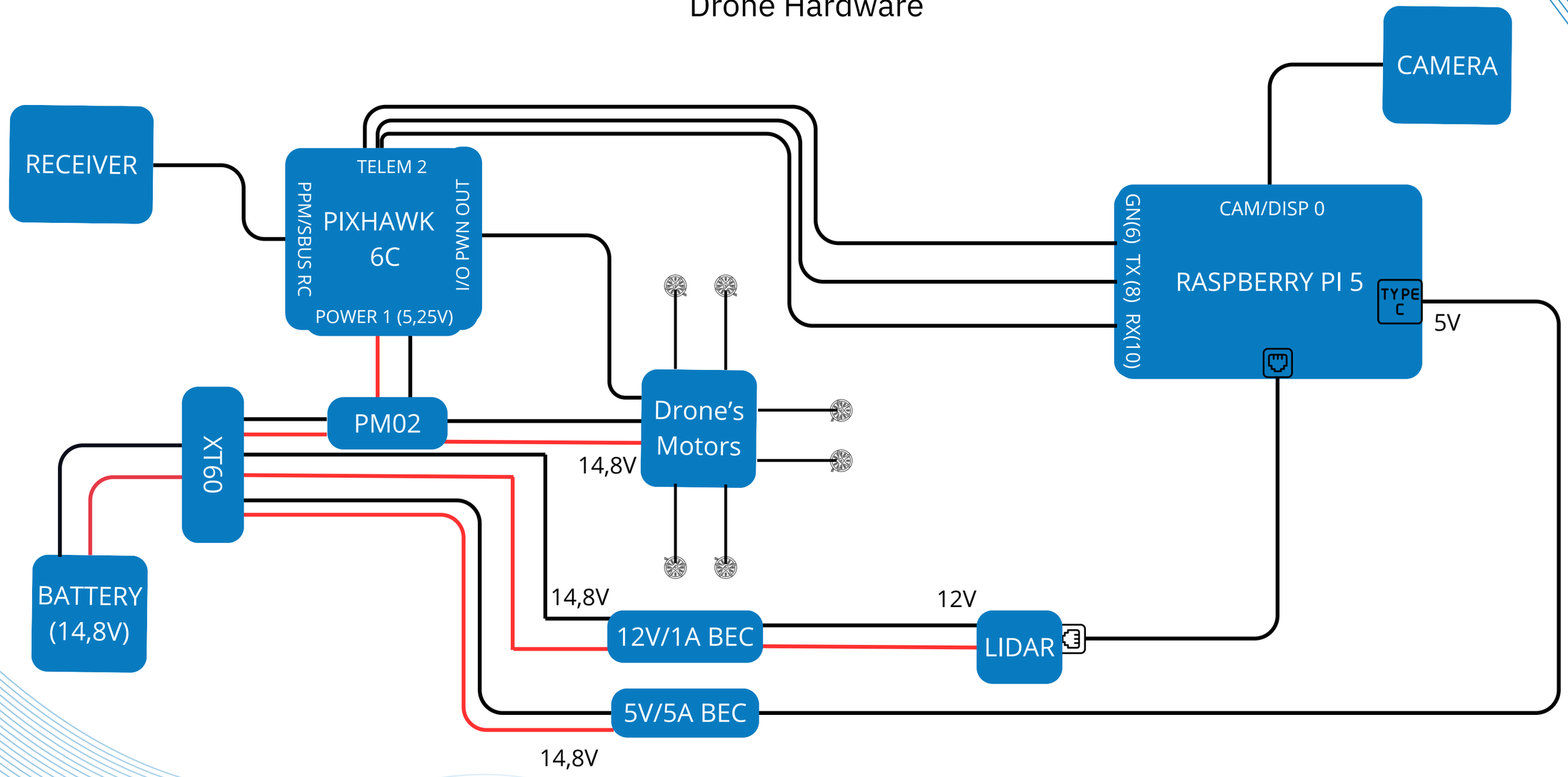
3. WP0: System Integration

ROS2 Graph-LiDAR



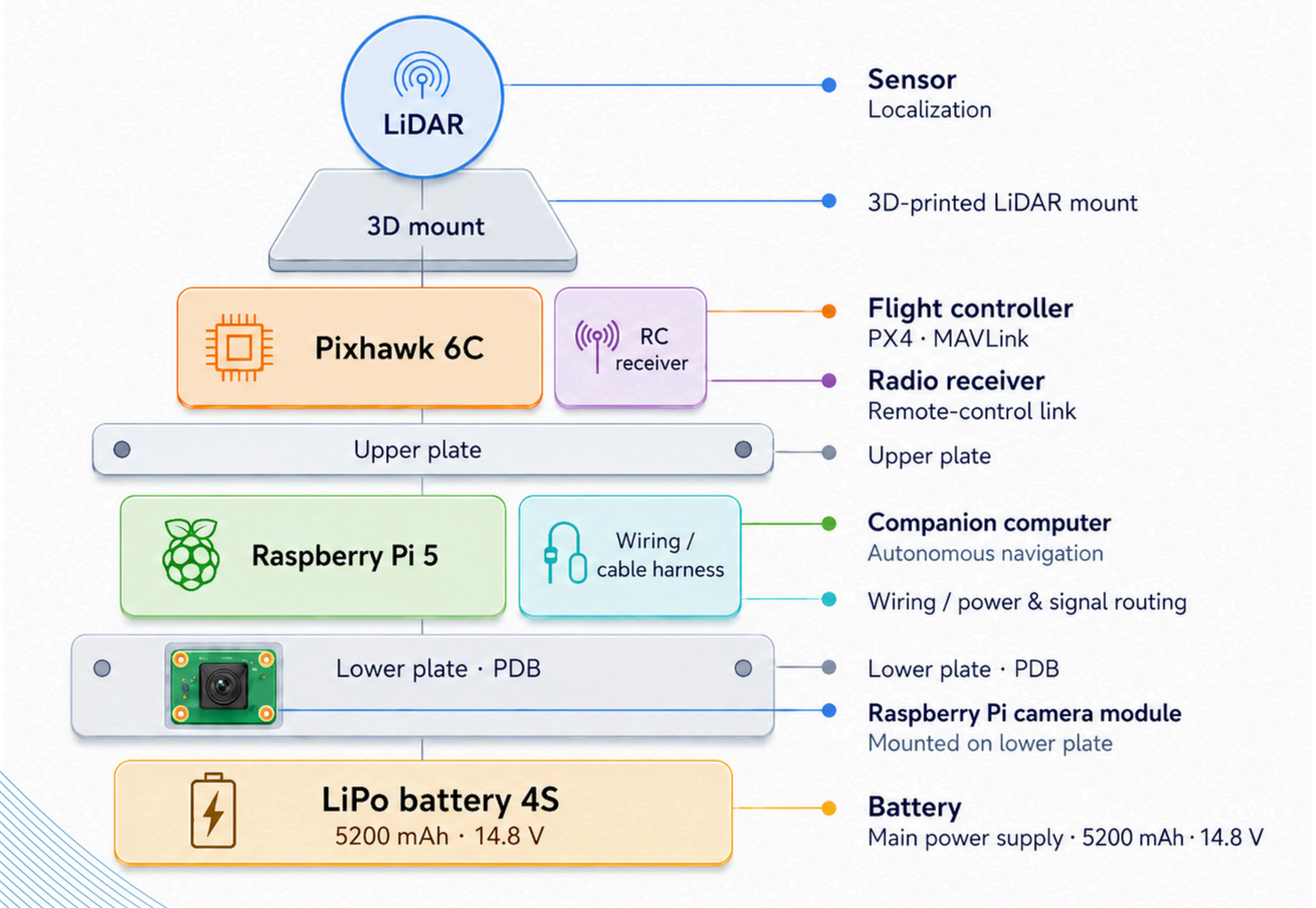
3. WP0: System Integration

Drone Hardware



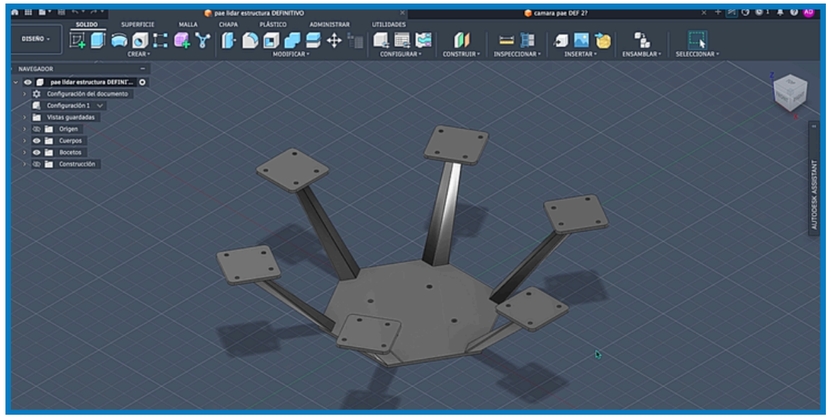
4. WP1: Drone control

Structure and organisation



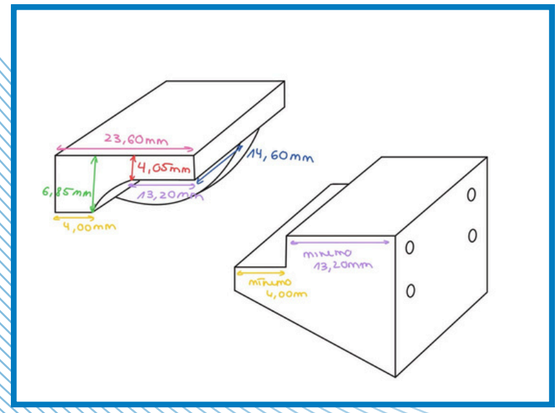
4. WP1: Drone control

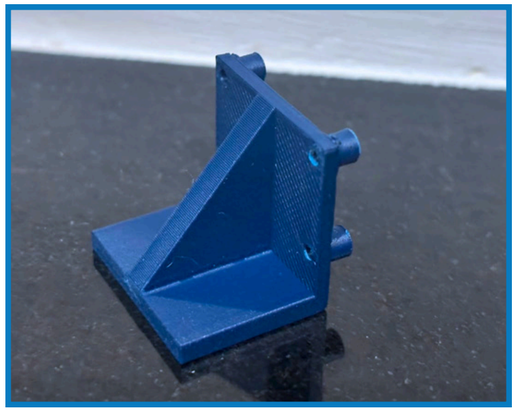
3D Printing

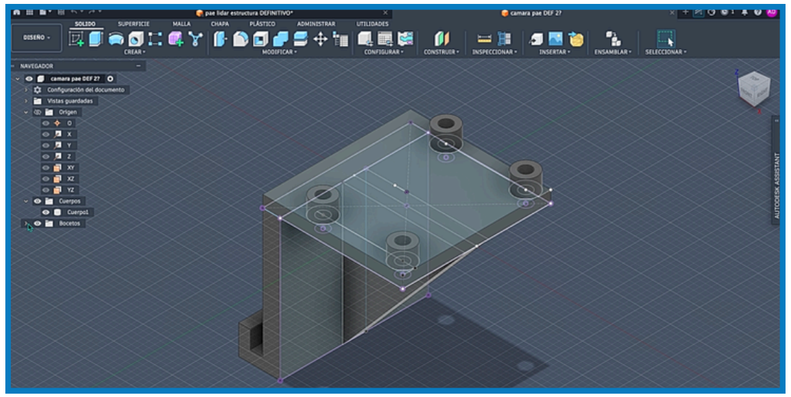


AUTODESK® FUSION 360™







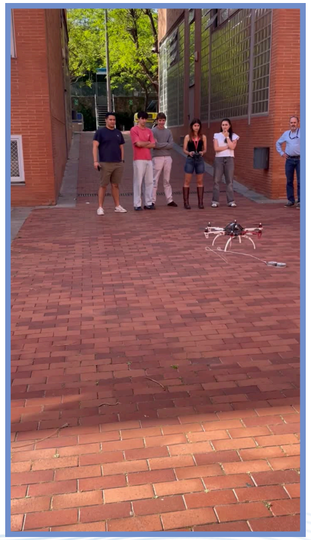


4. WP1: Drone control

Fight configuration and testing







QGroundControl

Flight Modes Setup

Flight Modes Setup is used to configure the transmitter switches associated with Flight Modes.

Flight Mode Settings

Mode Channel	Channel 5
Flight Mode 1	
Flight Mode 2	
Flight Mode 3	
Flight Mode 4	
Flight Mode 5	
Flight Mode 6	

Switch Settings

Arm switch channel	Unassigned	Landing gear switch channel	Unassigned
Emergency Kill switch channel	Channel 7	Loiter switch channel	Unassigned
Offboard switch channel	Unassigned	Return switch channel	Unassigned

Channel Monitor



4. WP1: Drone control

Next Steps: Autonomus flight and autonomy

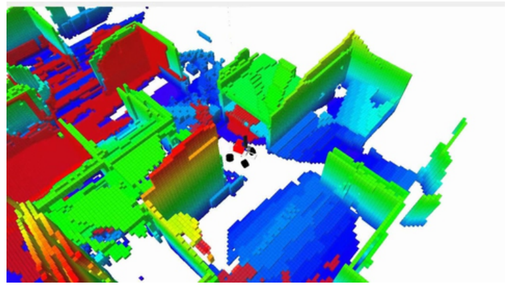
QgroundControl autonomous flight



Mavlink and SLAM autonomous flight

MAVLINK
MICRO AIR VEHICLE COMMUNICATION PROTOCOL

Raspberry Pi OS



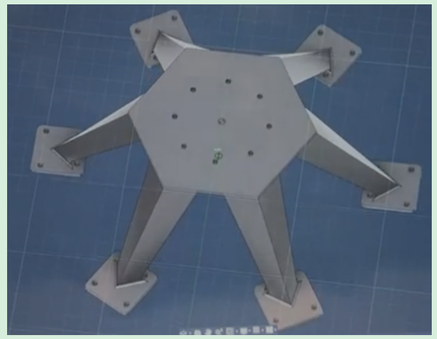
Battery Autonomy

- The drone weighs 2.2 kg
- 4S 5,200 mAh LiPo battery
- Drawing approximately 602 W (40.8 A) during hover
- Estimated **flight time of 6 to 7 minutes** under normal operating conditions.

5. WP1: Lidar

HARDWARE IMPLEMENTATIONS

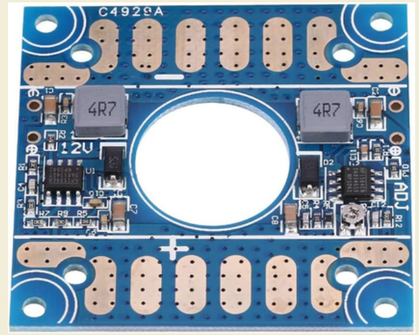
3D-Printed Mount



LiDAR-to-Raspberry Pi physical interfacing



Power cable modification and voltage regulator integration



5. WP1: Lidar

SOFTWARE ADAPTATION AND OPTIMIZATION

ROS 2 Jazzy Migration

Ported the Point-LIO and driver packages from ROS 2 Humble to Jazzy by updating the codebase to the C++17 standard.





Raspberry Pi 5 Optimization

Enabled multi-core processing and ARM architecture support to maximize the SLAM performance on the quad-core CPU.

Resource Management

We disabled the RViz visualizer by default to prioritize CPU cycles for the other tasks of our system.

5. WP1: Lidar

GOAL ASSESSTMENT: ACHIEVEMENTS & CHALLENGES

Real-Time Odometry & Orientation

We achieved the extraction of the drone’s position and orientation in real-time while in motion. The system can now track the drone's movement during flight.

Real-Time Mapping & App Visualization

The live map is directly visible through our application, allowing us to monitor the environment as the drone moves.

Real-Time Autonomous Flight Control

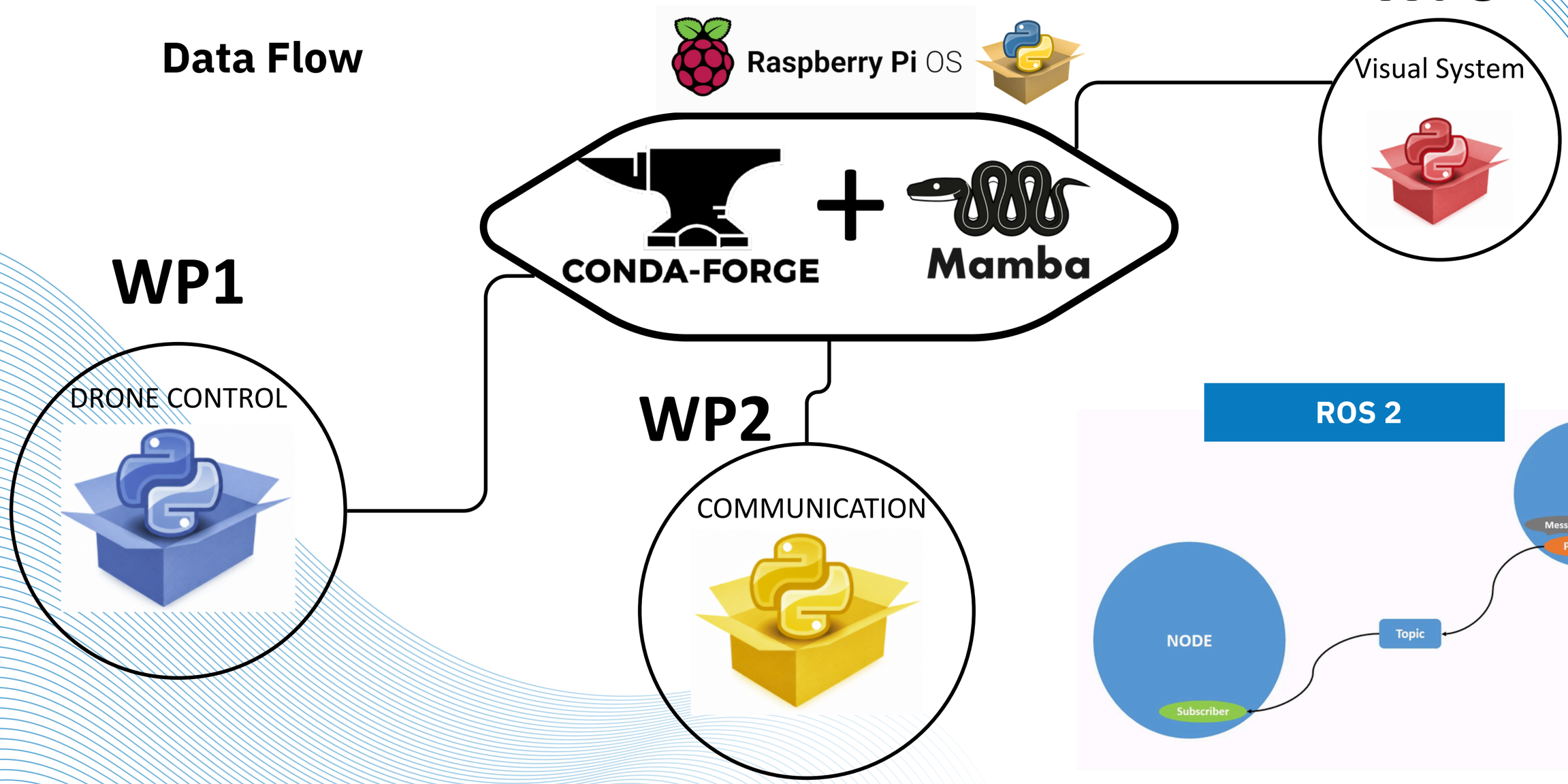
Integration of SLAM odometry into the flight controller for active navigation was not performed within the current project phase.

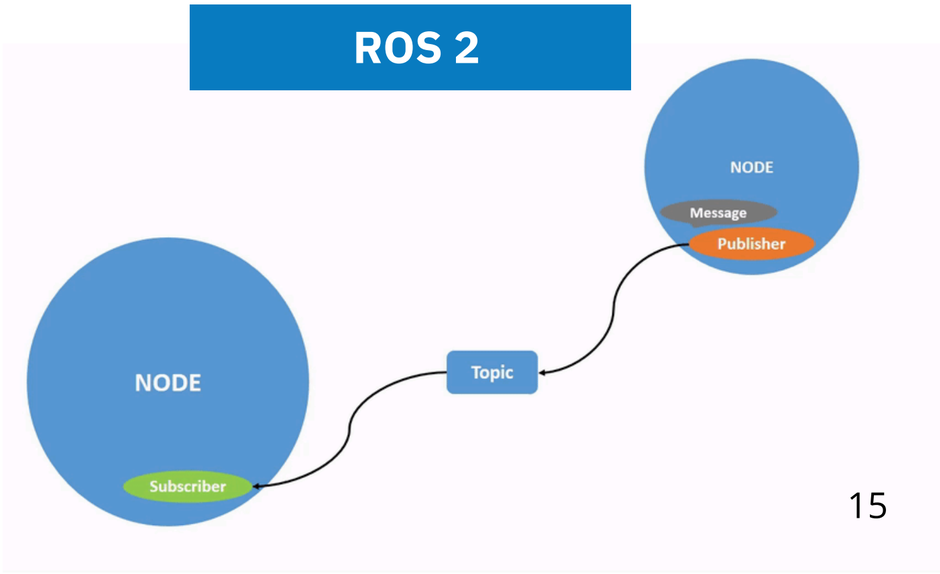
Active Obstacle Avoidance

Developing algorithms to use the 3D point cloud for real-time collision prevention remained outside the final scope.

6. WP2: Communication

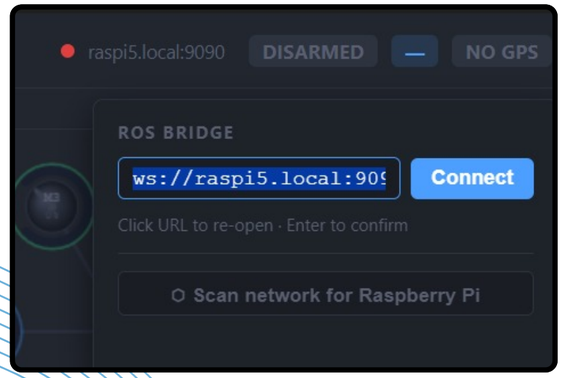
Data Flow



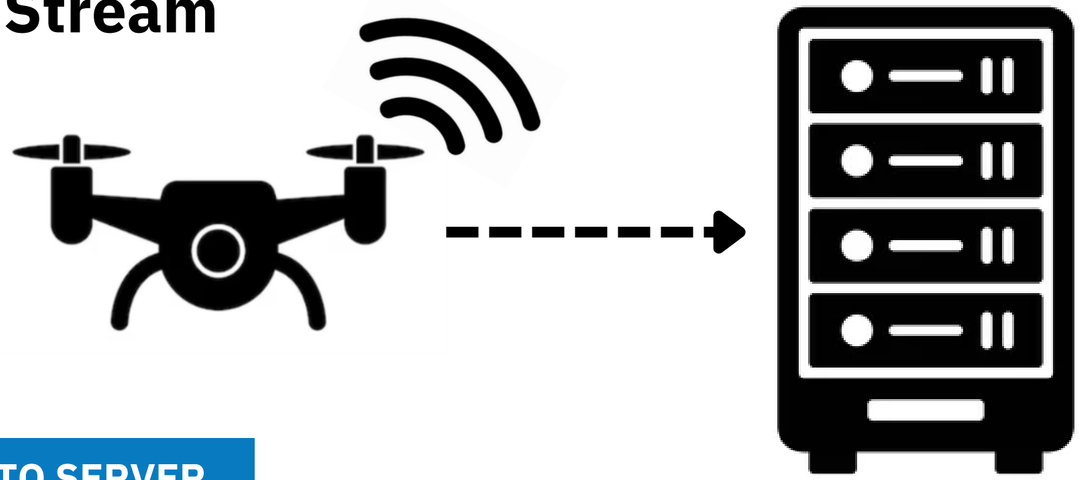


6. WP2: Communication

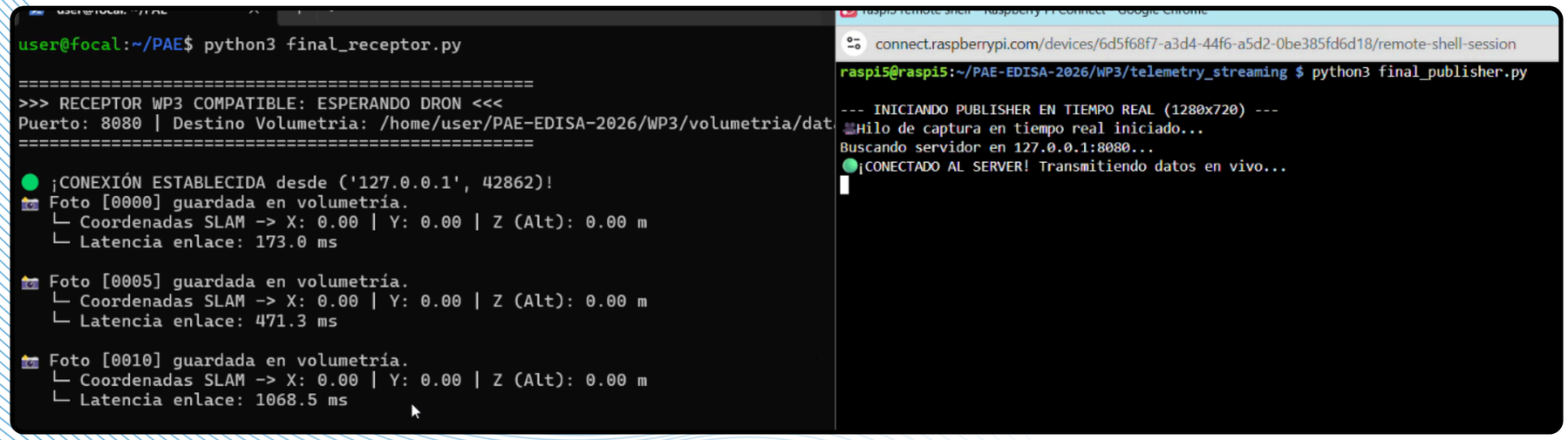
USER CONTROL



Data Stream



DRON TO SERVER

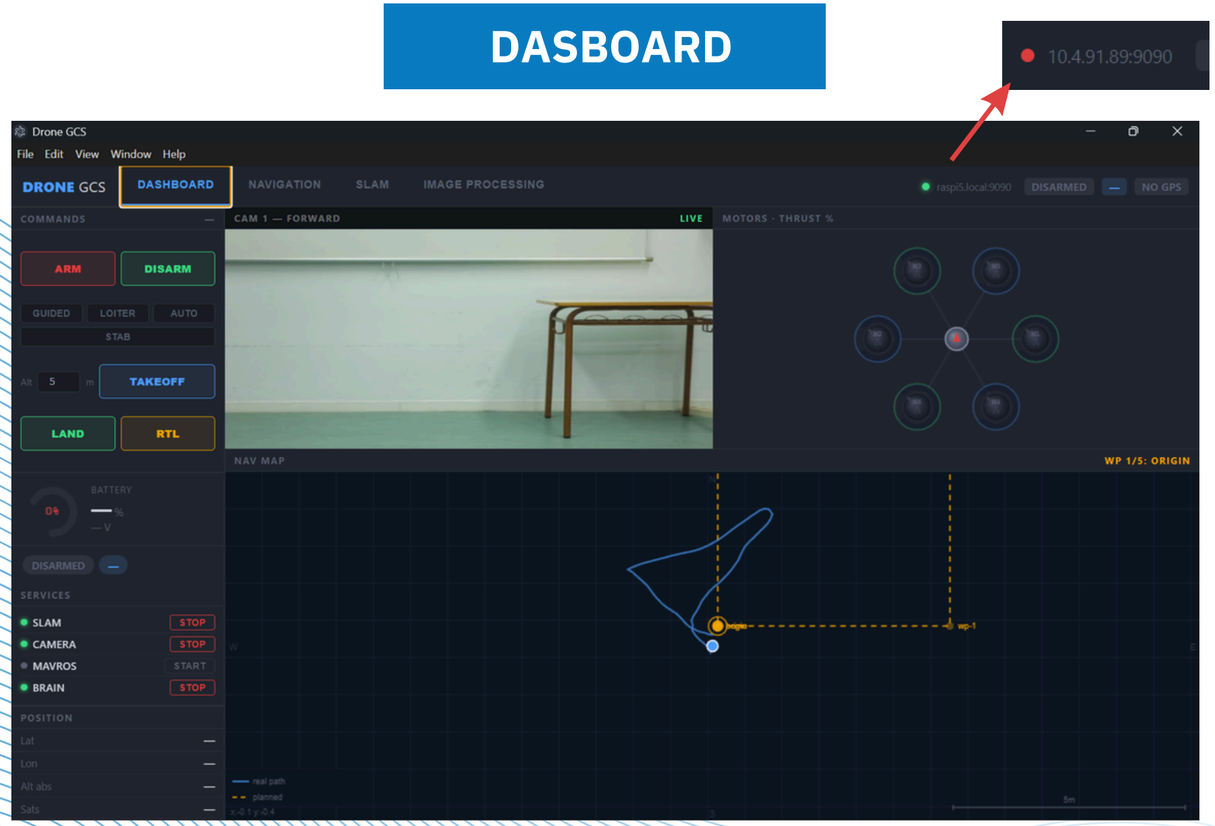


6. WP2: Communication

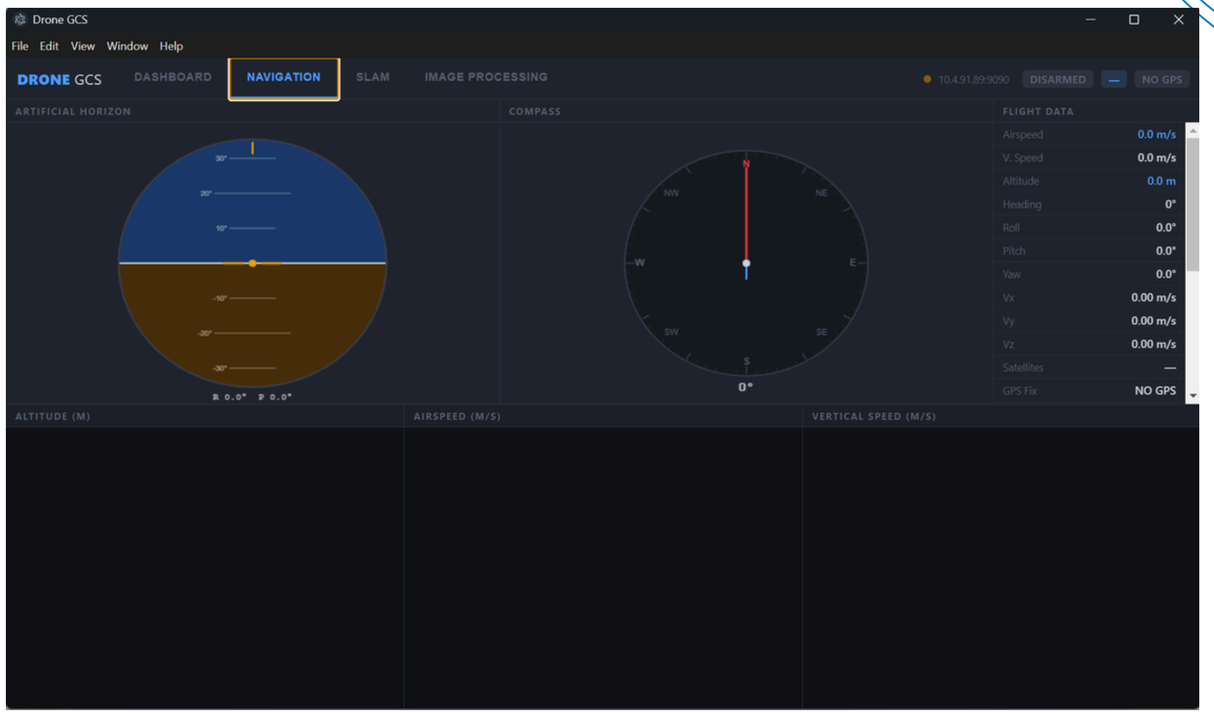
Drone GCS - Graphic user interface

The app is built with Electron, a framework based on Node.js. The same code works on Windows, Linux

DASBOARD



NAVIGATION



6. WP2: Communication

Drone GCS - Sistema de visualización

SLAM

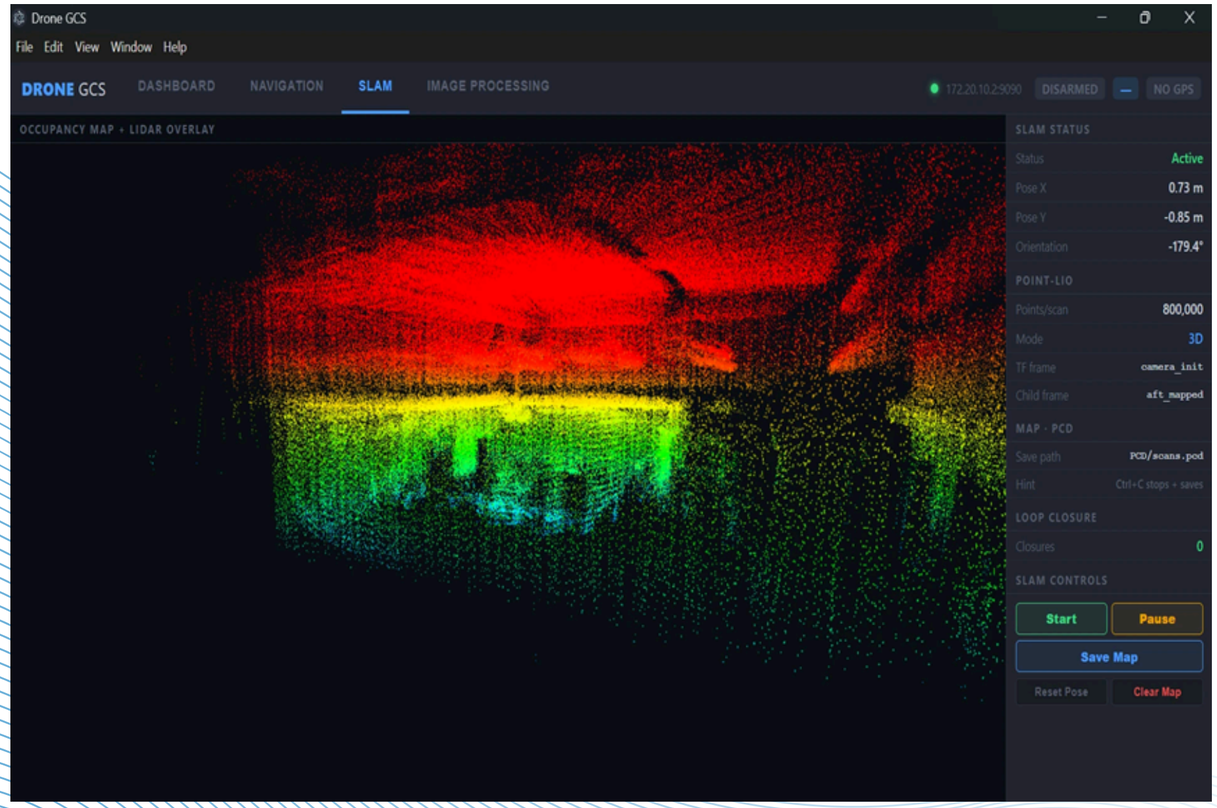
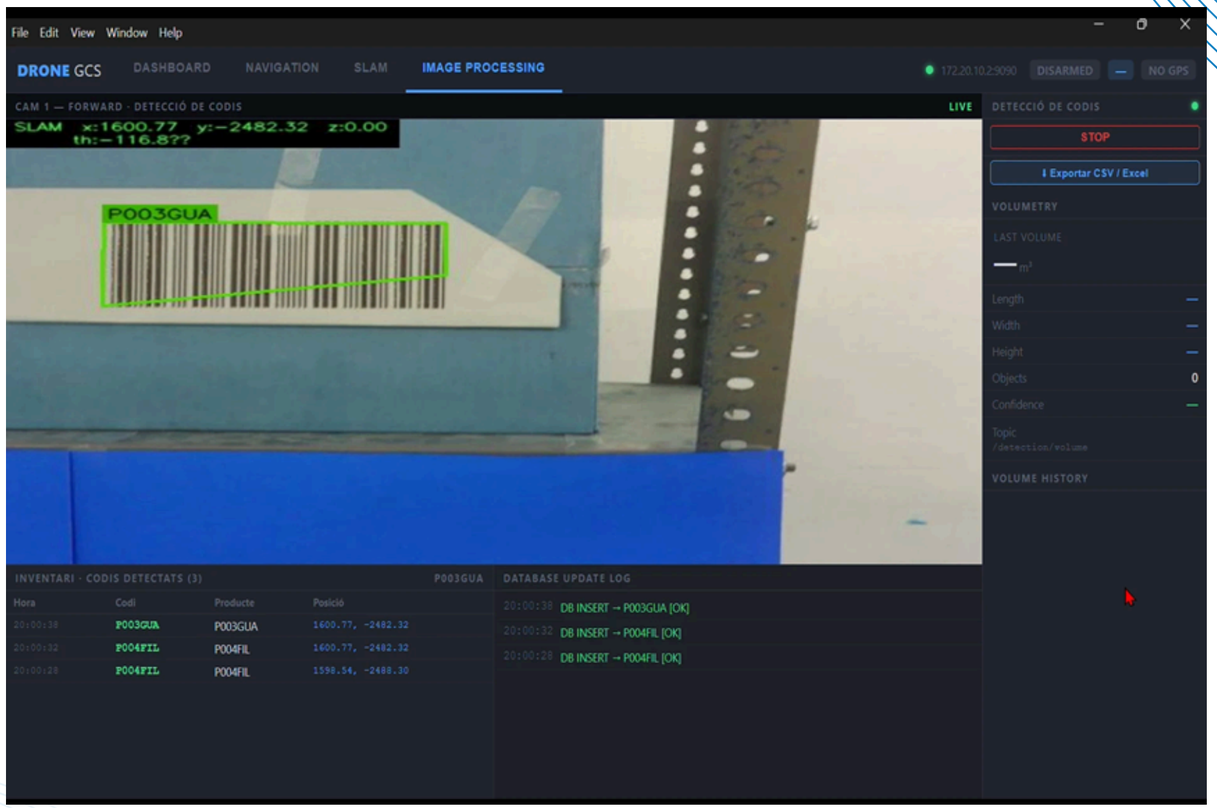


IMAGE PROCESSING



7. WP3: Visual System

From a simple system to the real warehouse



Dependency order is strict: BLOC0 orchestrates BLOC1 → BLOC2 → BLOC3

BLOC0.py — **Central Orchestrator** — CORE

Run whole flow · reads barcodes (pyzbar) · manages inventory state machine · handles code ↔ box association

BLOC1 — **Box Detection & Tracking** — *"What boxes exist, and where are they?"*

GroundingDINO detection → filter pipeline (beam zone, NMS…) → SAM segmentation per box

BLOC2 — **Vertex Detection** — *"Where are the corners?"*

SAM mask refinement (erode / close-H / bottom crop) → convexHull + approxPolyDP (5-6 verts)

BLOC3 — **Volumetry** — *"What are the dimensions & volume?"*

Calculates the size of the box: Frontal vs perspective branch

Shared models (loaded once in BLOC0) — GroundingDINO — MobileSAM

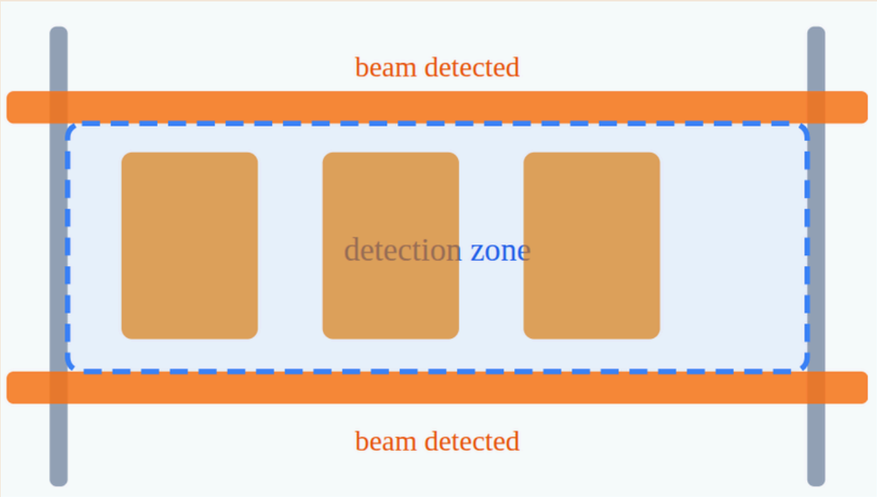
7. WP3: Visual System

Advances since the last update

1 Detection

Chain of filters:

beam_zone -> min_size -> aspect -> **colour** -> barcode -> containment -> dist_centres -> NMS-> pallets



Detects orange shelf beams by colour

2 Structure

Before: separate scripts

Now: all joined under the controller (unification)

Each box ↔ its code

RESULT: this box · this product · this shelf

```
==========================================
RESULTATS DE LA CAIXA [ID 2]
> Codi / Producte:   FILM ESTIRABLE
> Codi / Estanteria: 32041
==========================================
```

3 Reading codes

Failed: light · angle · blur

Detect box → zoom in → read clearly



7. WP3: Visual System

modular pipeline · one orchestrator + three functional blocks

BLOC 0 · Orchestrator

runs the pipeline per frame · manages inventory state · links each box to its barcode

BLOC 2 · Detection + Tracking

finds every box in the frame

- GroundingDINO (open-vocabulary detector)
- 8-filter cascade: beams → size → ratio → color → barcode → containment → distance → NMS
- SAM segmentation per surviving box
- centroid tracking with LiDAR-adjusted radius

BLOC 1 · Vertex Recovery

recovers the box corners

- SAM mask + 5 morphological cleaning ops
- iterative approxPolyDP → 5–6 vertices
- edge fitLine + intersection refinement
- QC: discard if any vertex > 25 px off contour

BLOC 3 · Volumetry

turns geometry into cm^3

- 4 vertices + 90° angles → frontal face (direct W · H)
- ≥ 5 vertices → parallax depth from LiDAR
- median across frames · frontal as ground truth
- output: width · depth · height · volume (cm^3)

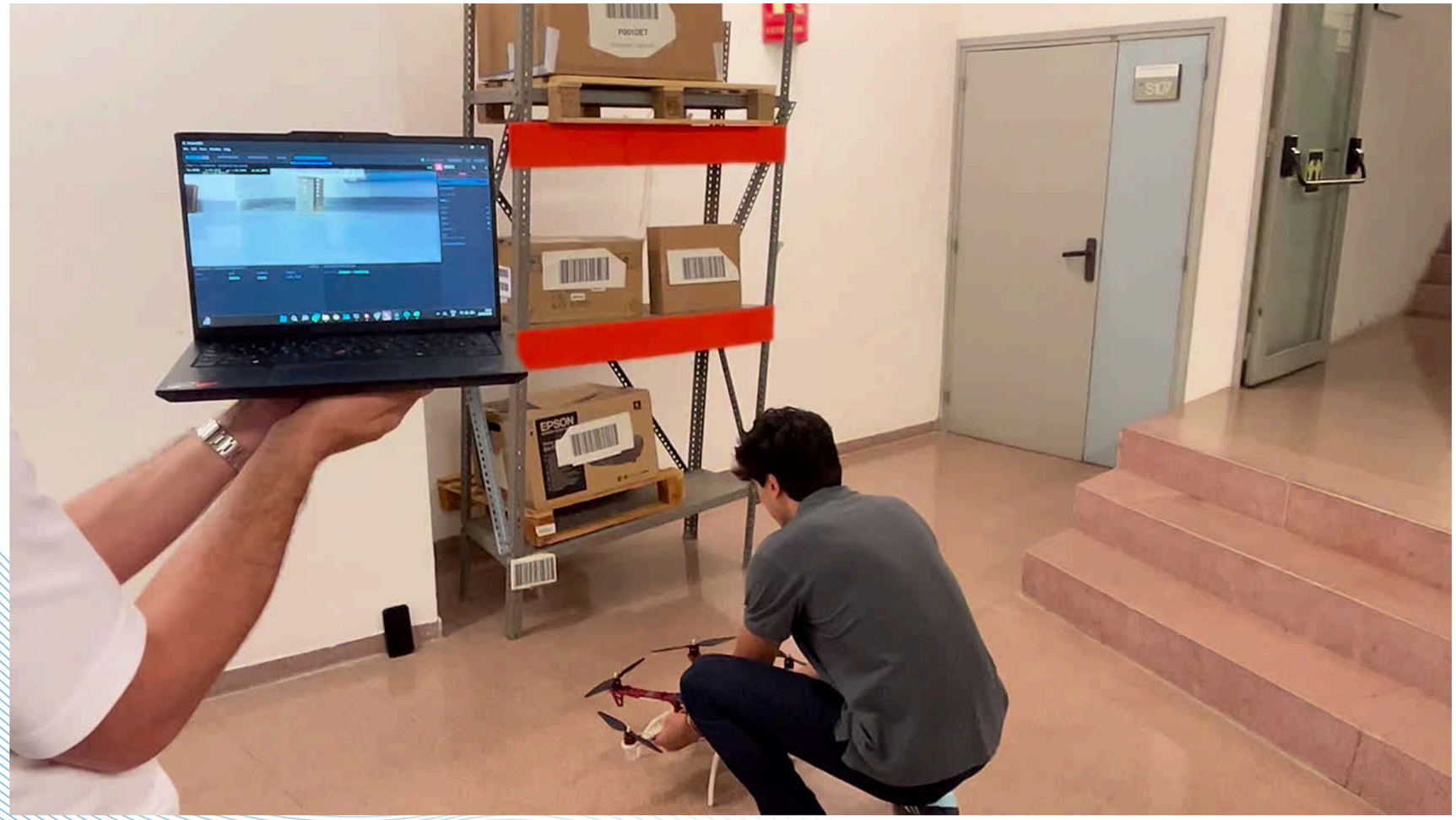
8. Conclusions

Project requirements conclusions:

- ≈ Autonomous flight
- ✓ Smart scanning and identification of labels in barcode or QR code format.
- ✓ Volumetric analysis to identify materials and estimate stock quantities.
- ✓ Inventory results sent to a database for subsequent analysis.
- ✓ Warehouse plan available showing the layout of the aisles and locations.
- ✓ Wi-Fi network available in the warehouse for data transmission.

10. Demostration

Video



Questions and contributions

Thank you so much